TECHNICAL DESIGN WHITEPAPER

Sentinel-Prime

An Agentic AI Cyber Resilience Platform for Critical National Infrastructure

Specialist Behavioural Detectors • Graph-Based Threat Correlation • MITRE ATT&CK Graph-RAG
Constrained Multi-Agent Reasoning • Adaptive Deception • Deterministic Policy-Gated Response

Prepared for the Economic Times AI Cyber Resilience Hackathon

Challenge 7 — AI-Driven Cyber Resilience for Critical National Infrastructure

July 2026

Version 3 — Specialist ML Detectors + LightGBM Meta-Correlation + Constrained AI Decision Reasoning + Adaptive Deception

Table of Contents

Abstract

Critical national infrastructure (CNI) operators face adversaries that deliberately operate below the detection threshold of signature-based tooling, and the resulting dwell time between initial compromise and discovery is routinely measured in weeks. Sentinel-Prime is an agentic AI cyber resilience platform that addresses this gap by combining domain-specialist machine learning detectors for network, identity, endpoint, and OT/ICS telemetry with a graph-based correlation layer, a MITRE ATT&CK-grounded retrieval-augmented reasoning stage, and a constrained multi-agent decision pipeline whose output is executed only through a deterministic policy gate. The architecture is deliberately partitioned so that statistical inference, semantic reasoning, and execution authority are handled by separate, independently auditable components: machine learning models detect measurable behavioural deviation, a knowledge graph encodes entity relationships and attack paths, large language model (LLM) agents reason over evidence under structural constraints, and a deterministic risk engine — never the language model — authorises containment. This document presents the system architecture, the engineering rationale behind each design decision, the current implementation status, and the evaluation methodology to be used for benchmarking, prepared for the Economic Times AI Cyber Resilience Hackathon (Challenge 7).

---

1. Problem Statement and Motivation

1.1 Challenge Context

The Economic Times AI Cyber Resilience Hackathon (Challenge 7 — AI-Driven Cyber Resilience for Critical National Infrastructure) frames the problem in terms of detection latency: CERT-In handled over 1.59 million cybersecurity incidents in 2023 alone, a volume that has continued to grow through 2024–25, and public-sector infrastructure — including AIIMS Delhi (2022 ransomware outage), CBSE examination systems (2024 data breach), and a coordinated 2026 attack disrupting CBSE board-examination infrastructure — has been repeatedly and visibly compromised. The stated root cause is architectural: over 70% of government entities operate end-of-life IT infrastructure, detection still depends heavily on known-signature matching, and advanced persistent threats (APTs) are explicitly engineered to stay below the noise floor of that matching process. The challenge statement calls for a platform that autonomously detects behavioural anomalies, correlates weak signals across heterogeneous IT and OT environments, maps attack progression against known threat frameworks, and orchestrates containment — compressing detection-to-response time from weeks to hours.

1.2 Why Signature and Single-Model Approaches Fail

- Signature and rule-based detection is reactive by construction: a technique must first be observed and catalogued elsewhere before it can be matched, which structurally cannot catch novel or low-and-slow tradecraft.
- SIEM platforms deployed without a correlation layer generate high alert volume with a high false-positive rate, producing analyst fatigue that itself becomes a detection-latency factor.
- Collapsing an incident into a single verdict discards competing explanations; an analyst, or an automated system, that commits early to one hypothesis has no structured way to weigh the operational cost of acting on a wrong one.
- IT and OT telemetry are heterogeneous in feature space, sampling rate, and failure semantics. A model trained on pooled, row-concatenated data across network flow, authentication, endpoint process, and industrial sensor domains is training on four different underlying distributions as if they were one, which degrades rather than improves detection quality.
- Even after a breach is confirmed, most SOC workflows have no systematic way to establish what else the attacker touched, and limited closed-loop visibility into whether a containment action actually worked.

1.3 Design Response

Sentinel-Prime's response is to keep statistical detection, semantic reasoning, and execution authority as separate, composable layers rather than a single end-to-end model. Each behavioural domain is served by a specialist detector trained only on data from that domain; detector outputs are normalised into a common schema and correlated on a knowledge graph; a constrained multi-agent reasoning pipeline consumes the correlated evidence together with retrieved MITRE ATT&CK context to build a cross-domain incident narrative and propose response actions; and a deterministic policy gate — not the language model — decides whether a proposed action is authorised for autonomous execution or must be escalated to a human. This separation of concerns is the platform's central design principle and is revisited throughout this document.

---

2. Design Goals and Non-Goals

2.1 Design Goals

- Behavioural, not signature-based, detection across IT and OT domains, using models appropriate to each domain's label availability and feature structure.
- Weak-signal fusion: no single detector score independently triggers containment; evidence must be corroborated across domains and graph context before an incident is escalated to AI reasoning.
- Grounded reasoning: AI agents reason against retrieved MITRE ATT&CK knowledge rather than open-ended judgment, and every hypothesis is traceable to evidence and technique identifiers.
- Self-correction before action: a dedicated critique stage adversarially tests the reasoning output before any containment action is proposed.
- Active uncertainty reduction: where confidence is moderate and a hypothesis is testable, the system gathers additional high-confidence evidence via deception rather than acting on incomplete information.
- Deterministic execution authority: whether an action executes autonomously depends on deterministic, auditable business logic — confidence and blast-radius thresholds — never on LLM output alone.
- Full auditability: every score, hypothesis, retrieved technique, decoy interaction, and action outcome is reconstructible from a tamper-evident record.
- Closed-loop improvement: incident outcomes feed back into detection baselines and defined re-entry points in the pipeline rather than being discarded after resolution.

2.2 Explicit Non-Goals

- Sentinel-Prime does not claim production readiness for live CNI deployment in its current hackathon-stage implementation; it is a lab-scale prototype exercised against public benchmark datasets and a controlled cyber range (Section 12).
- The system does not grant the language model direct execution authority over containment actions at any confidence level; this is an architectural invariant rather than a configurable option (Section 9).
- Cloud-native telemetry and containment (for example, AWS-specific integrations) are out of scope for the current implementation; the platform targets on-premise enterprise and ICS infrastructure (Section 14).

---

3. System Overview

3.1 Design Principle

```
ML detects measurable behaviour.
The graph connects entities and attack paths.
ATT&CK Graph-RAG retrieves threat knowledge before AI reasoning.
```

```
Constrained AI agents correlate, hypothesize, predict, test uncertainty, and plan
responses.
Deterministic policy — not the LLM — authorizes execution.
```

This five-line statement is the architectural contract that every component in Sentinel-Prime is required to honour. It fixes the division of labour between statistical models, graph structure, retrieval, language-model reasoning, and deterministic control, and it is the basis on which every design decision in the remainder of this document is justified.

3.2 End-to-End Pipeline

The platform is organised as a linear pipeline with defined feedback re-entry points, summarised in the seventeen runtime stages below and expanded in Sections 5–10. The canonical pipeline order is:

```
Telemetry
  ↓
Specialist ML Detectors
  ↓
Common Evidence Object
  ↓
Cyber Entity Graph
  ↓
LightGBM Meta-Classifier
  ↓
MITRE ATT&CK Hybrid Graph-RAG
  ↓
Analysis Agent
  ↓
Critique Agent
  ↓
Action Agent
  ↓
Adaptive Deception
  ↓
Risk Engine
  ↓
Deterministic Policy Gate
  ↓
SOAR
  ↓
Monitoring
  ↓
Audit Ledger
```

Figure 1. Sentinel-Prime end-to-end decision pipeline. [TODO: Insert rendered architecture diagram / Mermaid export]

#	Stage	Function
1	Telemetry	Endpoint, identity, network, application, and optional OT telemetry collected
2	SIEM normalisation	Wazuh + Elasticsearch normalises, indexes, and maintains rolling baselines
3	Behavioural sensing	Specialist ML models generate network, identity, endpoint, and OT evidence
4	Evidence normalisation	Detector outputs become the Common Evidence Object
5	Cyber graph update	Users, hosts, processes, IPs, and critical assets update the live Cyber Entity Graph
6	Statistical correlation	LightGBM meta-classifier combines detector, Sigma, graph, and deception evidence
7	ATT&CK Hybrid Graph-RAG	FAISS semantic retrieval plus ATT&CK knowledge-graph structural retrieval
8–10	AI reasoning (Analysis)	Cross-domain incident story, ranked hypotheses, attack-stage prediction

#	Stage	Function
11–12	Adaptive deception	Moderate-confidence incidents trigger graph-guided active hypothesis testing; interactions feed back into evidence
13	AI response planning	Action agent proposes ATT&CK-grounded, parameterised containment candidates
14	Risk and dry-run	Deterministic logic evaluates operational impact, dependencies, and blast radius
15	Policy authorisation	Allowlisted low-risk actions route to SOAR; high-impact actions route to human approval
16	Outcome monitoring	Resolved / persisted / escalated outcomes feed back into the pipeline
17	Audit and dashboard	Evidence, RAG context, hypotheses, predictions, deception activity, and actions are recorded

Table 1. The seventeen runtime stages of the Sentinel-Prime pipeline.

4. Specialist Behavioural Detection Layer

4.1 Engineering Rationale: Specialist Detectors over a Monolithic Model

An earlier design pooled Isolation Forest and XGBoost as global models over all datasets combined. This was revised because CSE-CIC-IDS2018 (network flow), LANL Auth (authentication), Splunk BOTS v3 / OTRF Mordor (endpoint), and HAI (OT/ICS) represent structurally different feature spaces — different units, different sampling cadence, and different notions of "normal." Row-wise merging such data forces a single model to learn a joint distribution that does not exist in the underlying systems, which degrades detection quality in every domain simultaneously. Sentinel-Prime instead trains one specialist detector per behavioural layer, each producing structured evidence rather than a final verdict; correlation across domains is deferred to a dedicated meta-classifier (Section 5) that operates on low-dimensional, already-summarised detector outputs, where cross-domain fusion is statistically well posed.

4.2 Datasets

Behaviour	Dataset	Why Chosen	Limitation
Network	CSE-CIC-IDS2018	Enterprise-like, labeled attacks, 80 CICFlowMeter features	Not rich in UEBA or OT signal
Identity / UEBA	LANL Auth	708M+ auth events; strong for user–host relationship learning	Authentication only — no endpoint or OT coverage
Endpoint / SIEM	Splunk BOTS v3 + OTRF Mordor	SOC investigation context plus adversarial host/network telemetry	BOTS is investigation/CTF-format data
OT / ICS	HAI	HIL-augmented ICS testbed (steam turbine, hydropower) relevant to power/industrial CNI	Sector-specific; no enterprise identity context
Threat knowledge	MITRE ATT&CK STIX 2.1	Tactics, techniques, groups, software, and mitigations for RAG and graph constraints	Knowledge base, not telemetry
Honeypot events	Self-generated (lab)	Canarytoken/Conpot logs triggered via Caldera or Atomic Red Team	Lab-generated, not field data

Table 2. Training and evaluation datasets by behavioural domain.

Given the limited public availability of Indian CNI-specific telemetry, Sentinel-Prime uses internationally recognised enterprise and ICS datasets for initial detector training and benchmarking. Deployment-specific behavioural baselines are intended to be learned from an organisation's own telemetry post-deployment, while synchronised multi-layer attack data for meta-classifier calibration is generated in an isolated cyber range rather than assumed to exist in the public data.

4.3 Model Selection

Component	Model	Why Selected
Network	LightGBM	CIC-IDS2018 is labeled tabular flow data; CPU-efficient and captures nonlinear feature interactions
Identity / UEBA	Isolation Forest	Complete labels for compromised identities are unavailable; learns unusual behavioural windows unsupervised
Endpoint / SIEM	LightGBM + Sigma	LightGBM learns process/log feature interactions; Sigma adds deterministic, explainable rule evidence
OT / ICS	Isolation Forest (+ optional TCN-AE)	Cheap normal-process baseline; temporal-convolutional autoencoder upgrade available if compute allows
Threat correlation	LightGBM meta-classifier	Combined evidence is low-dimensional tabular data; a fusion transformer would require synchronised training data that does not yet exist at scale

Table 3. Model selection per behavioural domain, with justification.

4.4 Detector Input/Output Contracts

Network Detector (LightGBM — CSE-CIC-IDS2018)

Input features: flow duration, forward/backward packet counts, packet-length statistics, flow bytes/packets per second, inter-arrival-time mean and standard deviation, TCP flag counts, active/idle means, destination port, and protocol.

```
{
  "network_score": 0.91,
  "attack_class": "infiltration",
  "attack_probabilities": {"benign": 0.03, "ddos": 0.02,
                           "botnet": 0.08, "infiltration": 0.87},
  "confidence": 0.94
}
```

Identity / UEBA Detector (Isolation Forest — LANL Auth)

Derived features: login-hour deviation, authentication frequency over 1h/24h windows, unique-host counts, new-host ratio, source-host change rate, destination fan-out, peer-group deviation, and time since last authentication.

```
{
  "identity_score": 0.88,
  "user": "U101",
  "new_hosts": 12,
  "unusual_relationships": ["U101 -> SERVER-92"],
  "lateral_movement_signal": 0.79,
  "confidence": 0.90
}
```

Endpoint / SIEM Detector (LightGBM + Sigma — BOTS v3 + Mordor)

Derived features: PowerShell execution, encoded-command usage, rare-process score, parent-process rarity, process-tree depth, Office child-process spawning, LSASS access, unsigned binaries, external connections following process start, and Sigma match count including critical matches.

```
{
  "endpoint_score": 0.97,
  "process_chain": ["WINWORD.EXE", "powershell.exe", "rundll32.exe"],
  "sigma_matches": ["Suspicious Office Child Process", "Encoded PowerShell"],
  "candidate_techniques": ["T1059.001", "T1218"],
  "confidence": 0.95
}
```

OT / ICS Detector (Isolation Forest — HAI)

Window features: sensor mean/standard-deviation/rate-of-change, pressure and flow rate change, setpoint deviation, actuator/pump/valve switch counts, sensor-correlation deviation, and control-process inconsistency.

```
{
  "ot_score": 0.96,
  "affected_variables": ["pressure", "flow", "valve_state"],
  "process_deviation": {"pressure": 6.2, "flow": -37.0},
  "confidence": 0.97
}
```

Each detector emits a structured evidence object rather than a binary verdict, deliberately preserving class probabilities, confidence, and domain-specific context so that downstream correlation can weigh partial or conflicting signals rather than losing that information at the point of detection.

---

5. Evidence Fusion: Common Evidence Object, Cyber Entity Graph, and Meta-Classifier

5.1 Why a Common Evidence Object

Each specialist detector emits a differently shaped output. Without a shared schema, every downstream consumer — the graph builder, the meta-classifier, and the AI agents — would need domain-specific parsing logic, which is brittle and hard to audit. The Common Evidence Object (CEO) is the normalisation boundary between machine learning detection and AI reasoning: every detector output is mapped into a single structure keyed by incident and entity, before anything downstream is allowed to consume it.

```
{
  "incident_id": "INC-102",
  "entities": {
    "users": ["U101"],
    "hosts": ["ENG-WS-01", "SERVER-07"],
    "ips": ["10.0.1.20", "10.0.2.17"],
    "ot_assets": []
  },
  "network": {"score": 0.82, "class": "infiltration"},
  "identity": {"score": 0.94, "new_hosts": 12},
  "endpoint": {
    "score": 0.97,
    "process_chain": ["WINWORD.EXE", "powershell.exe", "rundll32.exe"],
    "sigma_matches": ["Encoded PowerShell"]
  },
  "ot": {"score": 0.11},
  "deception": {"touched": false, "decoy_id": null},
  "candidate_techniques": ["T1059.001", "T1218"],
  "unified_threat_score": 0.91
}
```

Figure 2. Common Evidence Object schema — the ML/AI reasoning boundary.

5.2 Cyber Entity Graph

The Cyber Entity Graph (implemented in NetworkX, exposed as an incremental MultiDiGraph in the Unified Evidence Framework) represents users, hosts, processes, IPs, and critical assets as nodes, with observed interactions — CONNECTS_TO, RUNS_PROCESS, AUTHENTICATES_TO, CONTROLS — as edges. Graph analytics are used, not decorative: degree and betweenness centrality, PageRank, and modularity-based community detection produce attack-path, reachability, and blast-radius features that a purely tabular detector score cannot express, since compromise risk depends on an entity's position in the network topology as much as on its own behaviour.

Why graph analytics rather than flat feature engineering: lateral movement and blast radius are inherently relational properties — who can reach what, and along which path — and encoding them as hand-built scalar features would require re-deriving graph structure implicitly and imprecisely. A first-class graph representation keeps this reasoning explicit, inspectable, and reusable across the meta-classifier, the ATT&CK retrieval layer, and the deception placement logic.

5.3 LightGBM Meta-Classifier

The meta-classifier combines specialist detector scores, Sigma rule matches, graph-derived features (attack-path length, centrality, community-crossing), and deception evidence into a single unified threat score, trained on synchronised multi-domain incident scenarios generated in an isolated cyber range. LightGBM was selected over a learned fusion architecture (for example, a fusion transformer) because the combined evidence vector is low-dimensional and tabular by construction — the detectors have already done the representation learning within their own domains — and a transformer-scale fusion model would require volumes of synchronised, labelled, multi-domain incident data that do not yet exist publicly. LightGBM also preserves feature-importance-based explainability, which is required for the audit ledger (Section 11) and for justifying downstream containment decisions.

5.4 Threat-Score Routing

Unified Threat Score	Routing
< 0.40	Monitor / decay — no AI reasoning pipeline invoked, no dynamic honeypot
0.40 – 0.74	Full AI reasoning pipeline; if a testable, uncertain hypothesis exists, the Deception stage deploys a graph-guided decoy before response planning
≥ 0.75	Full AI reasoning pipeline; response planning proceeds immediately without waiting for honeypot confirmation

Table 4. Score-based routing from the meta-classifier into AI reasoning and deception.

---

6. MITRE ATT&CK Hybrid Graph-RAG

6.1 Placement in the Pipeline

Threat-knowledge retrieval is placed inside the decision pipeline, immediately before AI reasoning begins, rather than being attached after the language model has already formed a conclusion. This ordering matters: retrieval-after-generation can only be used to justify a decision post hoc, while retrieval-before-generation constrains the space of hypotheses the agent is allowed to reason within, which is what “grounded reasoning” (Section 2.1) requires in practice.

6.2 Offline Preparation

```
MITRE ATT&CK STIX 2.1
  ↓
```

```
Parse tactics, techniques, sub-techniques, groups, software, mitigations,
relationships
  ↓
Create ATT&CK documents (technique_id, name, tactic, description, platforms,
                        data_sources, detection_context, mitigations,
                        group_relationships, software_relationships)
  ↓
Sentence Transformer → Embeddings → FAISS Vector Index

STIX relationship objects → ATT&CK Knowledge Graph (NetworkX)
```

6.3 Runtime Retrieval Sequence

```
Common Evidence Object + Graph context + Meta-Classifier score
  ↓
Build retrieval query from observed evidence
  ↓
FAISS semantic retrieval  +  ATT&CK Knowledge Graph traversal
  ↓
Relevant ATT&CK context bundle
  ↓
Attach to AI decision context → AI Agents
```

6.4 Why Hybrid (Vector + Graph) Retrieval

Semantic vector search (FAISS over Sentence Transformer embeddings) answers "what ATT&CK knowledge is semantically similar to this evidence," which is well suited to matching free-text-like evidence (process names, command lines) to technique descriptions. It cannot, however, express structural relationships such as which techniques a given threat group is known to chain together, or which mitigations apply to a specific technique. The ATT&CK Knowledge Graph, built directly from STIX relationship objects and traversed with NetworkX, answers the structural half of the question. Using both retrieval modes together, rather than either alone, gives the reasoning agents both similarity-based and relationship-based grounding before they generate any hypothesis.

---

7. Constrained Multi-Agent AI Reasoning

7.1 Why Constrained Agents Rather Than a Single Open-Ended Prompt

Sentinel-Prime uses Gemini Flash as the underlying reasoning model for three logically separate, sequentially chained agent stages, each with its own restricted task, a structured JSON output schema, and a limited, purpose-built context window, rather than a single open-ended prompt asked to detect, explain, and respond in one pass. Constraining each stage's task and output shape serves two engineering purposes: it keeps each stage's failure mode narrow and testable in isolation, and it creates natural checkpoints — in particular the Critique stage — at which the reasoning can be adversarially validated before it is allowed to influence containment.

7.2 The Three-Stage Agent Pipeline

Agent	Stage	Input	Output	Why It Exists
Analysis	1	Evidence + graph + threat score + ATT&CK context	Cross-domain incident story; 2–4 ranked hypotheses (including a benign explanation); attack-stage prediction and candidate path	Consolidates correlation, hypothesis generation, and attack-progression prediction into a single structured reasoning pass, reducing API latency and cross-stage prompt complexity
Critique	2	Analysis Agent output plus	Validation verdict, critique	Acts as an adversarial

Agent	Stage	Input	Output	Why It Exists
		original context	feedback, corrected hypotheses where needed	reviewer, scrutinising the Analysis output for logical leaps, implausible ATT&CK technique attributions, or hallucinated evidence before any action is proposed
Action	3	Validated analysis, critique, ATT&CK mitigations, SOAR action allowlist	Ranked, parameterised containment/deception action candidates with supporting reasoning	Produces evidence-grounded response proposals; the agent only proposes exact, named function calls (e.g. isolate_host, block_ip, deploy_decoy) and never executes them directly

Table 5. The three sequentially chained Gemini Flash reasoning agents.

The Analysis stage's internal task — correlating cross-domain signals, ranking hypotheses, and predicting attack progression — corresponds to the Correlation, Hypothesis, and Prediction reasoning functions referenced in the pipeline's runtime-stage naming (Table 1, stages 8–10); these are implemented as one consolidated agent call rather than three separate API round-trips, which is a latency and prompt-complexity optimisation over an earlier design that chained them as fully separate agents.

7.3 Constraint Mechanisms

- Structured JSON output schemas per agent, validated before being passed to the next stage.
- Each hypothesis and predicted technique must reference specific evidence fields and ATT&CK technique identifiers rather than being expressed as free-form narrative only.
- The Action agent's proposal space is restricted to a pre-defined, named function allowlist (Section 9); it cannot propose an arbitrary or unparameterised action.
- Every agent call operates on a bounded context (the current incident's evidence and retrieved ATT&CK bundle), not the full historical corpus, limiting the scope for context-driven drift.

---

8. Adaptive Deception — AI-Driven Active Hypothesis Testing

8.1 Why Adaptive Deception

At moderate confidence (0.40–0.74 unified threat score), the system faces a genuine decision-theoretic problem: acting immediately risks disrupting legitimate operations on a false positive, while waiting passively risks losing the initiative to a genuine attacker. Adaptive deception resolves this by converting an untestable statistical hypothesis into a testable one: rather than asking a model to be more confident about ambiguous evidence, the system places a graph-guided decoy along the attacker's most likely predicted path and observes whether it is touched. A decoy interaction is treated as near-zero-false-positive evidence, because no legitimate user or process has a reason to access it.

8.2 Moderate-Score Deception Sequence

```
Score 0.40–0.74
  ↓
ATT&CK Graph-RAG → AI Analysis (correlation, hypothesis, prediction)
  ↓
Is there an uncertain and testable hypothesis?
  ├─ NO  → Monitor / analyst review
  └─ YES
       ↓
     Select hypothesis to test
```

```
        ↓
      Predict attacker action / target / path
        ↓
      Select decoy type (credential, SMB share, Conpot service, file)
        ↓
      Graph-guided safe placement
        ↓
      Deterministic deception-policy check
        ↓
      Deploy authorized decoy (hidden files: dot-prefix on Linux, attrib +H on
Windows)
        ↓
      Observe for fixed window (default 30 minutes)
        ↓
      Decoy touched?
```

If touched, the interaction is recorded, the Common Evidence Object and Cyber Entity Graph are rebuilt, the meta-classifier re-runs, and the full AI reasoning pipeline re-runs with high-confidence ground truth feeding directly into the Action agent. If not touched within the observation window, the deception evidence decays and the temporary decoy is removed — critically, an untouched decoy is not treated as evidence of benignity, only as an absence of confirming evidence, since a sufficiently cautious or slow attacker may simply not have reached the decoy yet.

Hypothesis	Predicted Action	Decoy Type	Placement
SMB lateral movement	Attacker enumerates shares	Fake SMB share	Graph-predicted path toward likely target
Credential discovery	Attacker tests privileged credentials	Canary credential / fake privileged artifact	Systems the entity has already accessed
OT discovery	Attacker probes industrial services	Isolated Conpot service	OT zone boundary

Table 6. Representative deception hypothesis-to-decoy mappings.

At high confidence (≥ 0.75), the pipeline does not wait for honeypot confirmation before proceeding to response planning, since the operational cost of delay outweighs the marginal value of additional confirming evidence at that confidence level.

---

9. Deterministic Risk Engine and Policy Gate

9.1 Why Execution Must Not Depend on the LLM

This is the architectural boundary that most distinguishes Sentinel-Prime from an “AI agent with tool access” pattern: the language model may propose a containment action, with reasoning, but it never decides whether that action executes. Business impact and blast radius are computed by deterministic Python/NumPy logic against configurable, per-organisation weights, and only that deterministic score — not any LLM-reported confidence — is compared against the policy threshold. This is a direct engineering response to the CNI context: an autonomous decision to isolate a host, revoke a credential, or block an IP on live infrastructure must be reproducible, explainable in terms an auditor can verify independently of the model, and immune to prompt-level or output-level manipulation of the model's own stated confidence.

9.2 Composite Risk Scoring

```
Composite Score = α × Containment Effectiveness  −  β × Business Impact
```

Both the containment-effectiveness estimate and the business-impact estimate are computed deterministically from graph topology, asset criticality configuration, and the dry-run simulation described below; α and β are

configurable per organisation or per asset-criticality tier, allowing a CNI operator to weight operational continuity against containment aggressiveness according to its own risk posture.

9.3 Dry-Run Simulation

Before any action is authorised, a dry-run simulator predicts which services, sessions, or dependent systems would be disrupted by executing it, using the Cyber Entity Graph's dependency structure. This converts “will this action break something important” from a question answered after the fact into a check performed before authorisation is granted.

9.4 Policy Gate

Condition	Routing
Confidence ≥ 0.75 AND low predicted blast radius	Auto-execute via SOAR playbook
Confidence < 0.75 OR high predicted blast radius OR critical/unsafe dry-run result	Route to human approval queue

Table 7. Deterministic policy-gate routing logic.

The 0.75 confidence threshold and the blast-radius sensitivity are deliberately implemented as configuration, not as constants embedded in agent prompts, so that the same reasoning pipeline can be tuned to a more conservative or more autonomous posture per deployment without touching the AI agents at all — reinforcing that autonomy level is a property of the deterministic layer, not the language model.

---

10. SOAR Orchestration and Autonomous Response

Actions authorised by the policy gate are executed through SOAR playbooks constrained to a pre-defined action allowlist — the same named, parameterised functions (isolate_host, block_ip, deploy_decoy, and equivalents) that the Action agent was restricted to proposing in Section 7.2. This closes the loop between what the agent is permitted to suggest and what the orchestrator is permitted to run: there is no path by which an unlisted or free-form action can reach execution, regardless of what the language model outputs.

Why SOAR rather than direct API calls from the agent layer: SOAR playbooks provide a natural point to enforce the action allowlist, attach dry-run and rollback semantics, and generate the structured execution record that the audit ledger consumes, none of which is naturally available if agents called target-system APIs directly.

---

11. Closed-Loop Outcome Monitoring and Audit Ledger

11.1 Outcome Feedback Loops

Loop	Trigger	Target	Effect
Deception confirmation	Adaptive decoy touched	Common Evidence Object + Cyber Entity Graph	Re-run full AI pipeline with high-confidence ground truth
Deception decay	Adaptive decoy expires untouched	Monitor queue	Remove decoy, decay evidence; do not mark as benign
Baseline update	Every resolved outcome	Baseline store	Updated rolling mean/standard deviation improves future detection
Persistence re-entry	Outcome = persisted	SIEM normalisation (stage 2)	Same entity re-enters detection with the same priority

Loop	Trigger	Target	Effect
Escalation re-entry	Outcome = escalated	Meta-classifier (stage 6)	Re-enters with elevated priority and broader permitted blast radius

Table 8. Feedback loops re-entering the pipeline after outcome monitoring.

Why explicit re-entry points rather than a generic retry: persisted and escalated outcomes have different operational meanings — persistence indicates the first action did not resolve the threat and detection should continue from telemetry, while escalation indicates the threat has grown in scope and correlation should re-run with elevated priority. Collapsing these into one generic retry path would lose that distinction and could under- or over-react to either case.

11.2 Tamper-Evident Audit Ledger

Every evidence object, retrieved ATT&CK context, hypothesis, critique verdict, proposed action, risk score, policy-gate decision, and outcome is recorded in a SHA-256 hash-chained audit ledger. This gives independent, model-agnostic traceability: an auditor can reconstruct why a given action was or was not authorised without needing to re-query the language model, and any retroactive modification of a ledger entry is detectable by hash-chain verification. Auditability of this kind is a direct requirement of deploying autonomous response in a CNI context, where containment decisions must be defensible after the fact to regulators and incident-response stakeholders.

---

12. Implementation Status

The following reflects the state of the repository as supplied; components not listed as implemented should be read as designed but not yet built, consistent with a hackathon-stage prototype.

12.1 Unified Evidence Framework (UEF) and Cyber Knowledge Graph

- BaseEvidence and specialist schemas: a dataclass model standardising inputs from network, identity, endpoint, and OT sensors, including validation constraints and contract checks.
- Evidence bus: a thread-safe, in-memory streaming bus with validation, normalisers, a SHA-256 duplicate cache, and a priority/timestamp-ordered event queue.
- Cyber Knowledge Graph: incremental graph storage wrapping a NetworkX MultiDiGraph, representing entity nodes (HOST, USER, PROCESS, PLC) and interaction edges (CONNECTS_TO, RUNS_PROCESS, AUTHENTICATES_TO, CONTROLS), with degree/betweenness centrality, PageRank, and modularity community metrics computed for AI reasoning.
- Correlation context builder: extracts bounded-hop-radius local subgraphs, generates natural-language summaries, orders timelines chronologically, and compiles structured CorrelationContext objects for agent prompt injection.

Directory structure

```
core/evidence/   Schemas, validators, normalizers, publishers, subscribers, queue,
cache, bus
core/graph/      Extractor, node/edge builders, indexing, queries, metrics, manager
core/context/    Context schema, timelines, summaries, builders
core/framework.py  Central facade coordinating the pipeline
```

12.2 AI Reasoning Core (Implemented)

- Hypothesis generation (agent/hypothesis_agent.py): takes the enriched alert, queries the FAISS index built by agent/rag/build_index.py, and uses Gemini Flash to generate 3–4 ranked hypotheses, including one benign explanation.

- APT attribution and blast radius (agent/apt_attribution.py): attributes activity to a threat actor and computes blast radius using a NetworkX topology loaded dynamically from config/topology.yaml.
- Risk scoring (risk_scoring/scorer.py): evaluates operational impact using the deterministic composite-score formula (Section 9.2), with configurable weights from config/risk_params.yaml.
- Pipeline integration (agent/pipeline.py): wires these modules together behind a single entry point, run_pipeline(evidence: dict) -> dict, consumed by the FastAPI/Flask dashboard backend.

12.3 Known Gaps to Production Architecture

The current repository uses a one-shot demonstration wrapper (run_phase1.py) and a basic check-status monitor rather than a continuously running service. Two upgrades are required to move from demonstration to production:

Continuous execution

- Enterprise standard: Apache Kafka, with the SIEM publishing normalised events to a siem-alerts topic and a fleet of consumer.py workers continuously pulling events into Phase1Pipeline.process().
- Lightweight alternative: a FastAPI server exposing a /api/v1/telemetry/ingest webhook, with the SIEM configured to POST events in real time as anomalies occur.

True closed-loop outcome monitoring

- The current monitor.py verifies only that a SOAR command executed without an API error; it does not verify ground-truth outcome.
- State management: incidents should transition from ACTIVE to VERIFICATION_PENDING after a SOAR action, rather than immediately to RESOLVED.
- Asynchronous verification: a task queue (Celery or APScheduler) schedules a verification task, for example five minutes after execution, that queries the SIEM/EDR directly for evidence the malicious behaviour has stopped; if it persists, the incident is marked PERSISTING and either triggers a more aggressive playbook or escalates to a human.

---

13. Technology Stack

Layer	Technology
IT telemetry	Sysmon, auditd, application/system logs
Network telemetry	Suricata, Zeek, firewall, DNS, NetFlow
Identity telemetry	Active Directory, VPN, authentication logs
OT telemetry	Historian exports, Modbus telemetry, sensor/actuator events
SIEM	Wazuh + Elasticsearch
Network ML	LightGBM
Identity / UEBA	Isolation Forest (LSTM autoencoder optional upgrade)
Endpoint detection	LightGBM + Sigma
OT anomaly detection	Isolation Forest (TCN autoencoder optional upgrade)
Threat correlation	LightGBM meta-classifier
Deception	Custom honeytokens (hidden files), Conpot (OT), controlled SMB/SSH decoys
Threat knowledge	MITRE ATT&CK STIX 2.1
Vector database	FAISS

Layer	Technology
Embeddings	Small pretrained Sentence Transformer
AI decision layer	Gemini Flash — 3 constrained agents (Analysis → Critique → Action)
Graphs	NetworkX (Cyber Entity Graph and ATT&CK Knowledge Graph)
Risk scoring	Deterministic Python/NumPy
Orchestration	SOAR playbooks with action allowlist
Audit ledger	SHA-256 hash chain
Dashboard	React SPA + Flask API + WebSocket/SSE
Preprocessing	pandas, scikit-learn, LightGBM, imbalanced-learn

Table 9. Full technology stack by architectural layer.

---

14. Deployment Architecture

The platform is containerised via docker-compose.yml to support scalable, continuous execution. All core microservices use restart: unless-stopped policies with internal healthchecks.

Service	Description	Port
elasticsearch	Primary SIEM datastore for normalised alerts and events	9200
api	Flask API serving dashboard data and executing SOAR/orchestration	5000
frontend	React SPA dashboard (Vite dev server or Nginx production build)	5173
webhook	Receiver for passive honeytoken alerts and deception interactions	8080
ml_worker	Background worker running specialist detectors and the meta-classifier	—

Table 10. Docker Compose service topology.

Cloud-native telemetry and containment (for example, AWS-specific hooks) are explicitly out of scope for the current implementation, which targets on-premise enterprise and ICS infrastructure; a future iteration would add cloud-native honeytokens, cloud SIEM integration, and cloud-native containment actions.

---

15. Evaluation Methodology

This section defines the evaluation protocol Sentinel-Prime is designed to be benchmarked against. In keeping with the current implementation status (Section 12), no benchmark results have yet been produced; all numerical fields are left as explicit placeholders to be populated once the benchmarking suite (scripts/eval/) has been run to completion against data/eval_ground_truth.json. This section also serves as the methodological specification for that run.

15.1 Detection Performance

Detection Rate (Recall) and False Positive Rate — per detector

Objective: Quantify each specialist detector's ability to identify true attack behaviour in its own domain while bounding nuisance alerts.

Purpose: Establishes whether each specialist model meets the accuracy/noise trade-off required before its output is trusted by the meta-classifier.

Measurement method: Evaluate each detector independently against its held-out test split; report recall, false-positive rate, precision, and F1 per attack class.

Dataset: CSE-CIC-IDS2018 (network), LANL Auth (identity), Splunk BOTS v3 + OTRF Mordor (endpoint), HAI (OT)

Baseline: Manual SOC triage baseline / published dataset baselines where available

```
Recall = TP / (TP + FN);  FPR = FP / (FP + TN)
```

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

Precision, Recall, F1 Score — meta-classifier

Objective: Assess the LightGBM meta-classifier's ability to correctly fuse multi-domain evidence into a unified threat verdict.

Purpose: The meta-classifier is the statistical gate into AI reasoning; its precision/recall trade-off directly determines how often the AI pipeline and deception layer are invoked unnecessarily or missed.

Measurement method: Evaluate on synchronised, labelled cyber-range incident scenarios not used in training.

Dataset: Isolated cyber-range synchronised attack dataset

Baseline: Individual specialist-detector scores used without fusion

```
F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

ROC-AUC

Objective: Summarise detector/meta-classifier discrimination ability across all thresholds.

Purpose: Provides a threshold-independent view of model quality, complementing the fixed-threshold precision/recall figures.

Measurement method: Compute ROC curve and AUC per detector and for the meta-classifier on held-out data.

Dataset: Same as corresponding detector/meta-classifier dataset above

Baseline: Random classifier (AUC = 0.5)

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

15.2 Threat Attribution and Reasoning Quality

ATT&CK Attribution Accuracy (Top-1 and Top-3 Technique Accuracy)

Objective: Measure whether the AI Analysis agent's predicted MITRE ATT&CK technique(s) match ground truth.

Purpose: Directly evaluates the Graph-RAG plus Analysis-agent chain against the hackathon's stated evaluation focus on APT attribution accuracy at the technique level.

Measurement method: Compare the agent's top-1 and top-3 predicted technique IDs against the ground-truth technique used to generate each simulated incident (via Caldera or Atomic Red Team).

Dataset: Caldera / Atomic Red Team-generated ground-truth scenarios

Baseline: Keyword/string-matching baseline against ATT&CK technique descriptions

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

Hypothesis Ranking Accuracy

Objective: Measure how often the correct explanation for an incident is ranked highest among the Analysis agent's 2–4 candidate hypotheses.

Purpose: Validates that the ranked-hypothesis output is actionable rather than merely plausible-sounding.

Measurement method: Compare the top-ranked hypothesis against ground-truth incident cause across the evaluation scenario set.

Dataset: Isolated cyber-range synchronised attack dataset

Baseline: Random ranking among generated hypotheses

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

15.3 Response Automation and Timeliness

Automation Coverage

Objective: Measure the proportion of containment steps that execute autonomously through the policy gate without requiring human approval.

Purpose: Directly evaluates the hackathon's stated evaluation focus on incident-response automation coverage, and quantifies the deterministic policy gate's real-world autonomy rate.

Measurement method: Percentage of proposed Action-agent steps that are auto-authorised by the policy gate versus routed to human approval, across the evaluation scenario set.

Dataset: Isolated cyber-range synchronised attack dataset

Baseline: N/A (baseline is 0% automation under fully manual SOC response)

```
Automation Coverage = Auto-authorised steps / Total proposed steps
```

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

Blast Radius Prediction Accuracy

Objective: Assess whether the dry-run simulator's predicted service/session disruption matches actual disruption observed when an action executes in the cyber range.

Purpose: Validates the risk engine's dependency model, which the policy gate relies on for the low-blast-radius auto-execution condition.

Measurement method: Compare dry-run predicted impact set against observed impact set after action execution in a controlled range run.

Dataset: Isolated cyber-range action-execution logs

Baseline: No-simulation baseline (action executed without dry-run check)

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR)

Objective: Quantify end-to-end latency from initial telemetry indicating compromise to detection, and from detection to authorised containment.

Purpose: Directly addresses the hackathon's core objective of compressing detection-to-response time from weeks to hours.

Measurement method: Timestamp differencing between injected compromise, meta-classifier alert, and policy-gate authorisation, averaged across the evaluation scenario set.

Dataset: Isolated cyber-range synchronised attack dataset

Baseline: Simulated manual SOC triage timeline

```
MTTD = t(alert) − t(compromise);  MTTR = t(contained) − t(alert)
```

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

End-to-End Pipeline Latency

Objective: Measure wall-clock time from telemetry ingestion through policy-gate decision for a single incident.

Purpose: Establishes whether the reasoning pipeline (retrieval + three agent calls) is fast enough for near-real-time operation.

Measurement method: Instrument each pipeline stage and record per-stage and total latency across repeated runs.

Dataset: Isolated cyber-range synchronised attack dataset

Baseline: N/A

Result: [TODO: Insert measured value]

Discussion: [TODO: Evaluation discussion — error analysis, comparison to baseline, failure modes]

15.4 Reporting Artifacts

- [TODO: Confusion Matrix — per detector and meta-classifier]
- [TODO: ROC Curve — per detector and meta-classifier]
- [TODO: Precision-Recall Curve — per detector and meta-classifier]
- [TODO: Benchmark Table — consolidated metrics across all detectors and the meta-classifier]
- [TODO: Benchmark Graph — MTTD/MTTR versus simulated manual-SOC baseline]
- [TODO: Evaluation Discussion — consolidated error analysis and threats to validity]

15.5 Evaluation Tooling

The evaluation suite is located in scripts/eval/. generate_synthetic_benchmark.py creates realistic APT incident scenarios spanning IT and OT telemetry; specialised evaluation modules test detector accuracy, ATT&CK attribution against the Graph-RAG layer, SOAR risk metrics, and audit-ledger immutability; and evaluate_all.py orchestrates the full suite against the pre-compiled ground truth in data/eval_ground_truth.json, measuring true positives, false positives, and MTTD against a manual-baseline comparator.

16. Hackathon Alignment

Sentinel-Prime maps directly onto the four illustrative build areas and the suggested technology list in the Challenge 7 problem statement, and onto its stated judging criteria.

16.1 Mapping to Suggested Build Areas

Challenge Area (as stated)	Sentinel-Prime Component
Behavioural Anomaly Detection Engine	Specialist network/identity/endpoint/OT detectors (Section 4) building per-entity behavioural baselines and continuously scoring deviation, without signature dependence
APT Campaign Attribution & Prediction Agent	MITRE ATT&CK Hybrid Graph-RAG (Section 6) plus the Analysis agent's attack-stage and technique prediction (Section 7)
Autonomous Incident Response Orchestrator	Deterministic risk engine, policy gate, and SOAR playbook execution with human escalation above blast-radius thresholds (Sections 9–10)
Government Infrastructure Vulnerability Prioritisation	Out of current scope; the Cyber Entity Graph's asset-criticality and topology model (Section 5.2) is the natural extension point for CVE-feed-driven prioritisation (Section 17)
Cyber Resilience Digital Twin	The dry-run simulator (Section 9.3) provides attack-path and impact modelling on the live Cyber Entity Graph without touching production systems, though full digital-twin scenario testing is future work (Section 17)

Table 11. Mapping from the hackathon's illustrative build areas to implemented Sentinel-Prime components.

16.2 Mapping to Suggested Technologies

- Agentic AI / multi-agent systems — the three-stage Analysis → Critique → Action Gemini Flash pipeline (Section 7).
- Unsupervised anomaly detection (UEBA) — Isolation Forest identity and OT detectors (Section 4.3).
- Graph AI (attack-path analysis, lateral-movement detection) — the Cyber Entity Graph and its centrality/community/blast-radius features (Section 5.2).
- RAG over threat intelligence and CVE databases — the MITRE ATT&CK Hybrid Graph-RAG (Section 6); CERT-In advisory and CVE-feed ingestion are noted as future extensions (Section 17).
- Knowledge graphs (MITRE ATT&CK TTP mapping) — the ATT&CK Knowledge Graph built from STIX relationships (Section 6.2).
- SOAR integration and response automation — Sections 9–10.

16.3 Mapping to Judging Criteria

Criterion	Weight	Primary Supporting Sections
Innovation	25%	Specialist-per-domain detection with graph-mediated fusion (Section 4–5); constrained three-stage agent reasoning with an adversarial critique step (Section 7); deception used as an active hypothesis-testing mechanism rather than passive monitoring (Section 8)
Business Impact	25%	MTTD/MTTR compression objective (Section 15.3); deterministic policy gate calibrated to organisational risk appetite (Section 9)
Technical Excellence	20%	Domain-appropriate model selection with explicit justification (Section 4.3); hybrid vector plus graph retrieval (Section 6.4); full audit-ledger traceability (Section 11.2)
Scalability	15%	Containerised microservice deployment (Section 14); documented path from demonstration wrapper to Kafka/FastAPI continuous ingestion (Section 12.3)
User Experience	15%	React SPA dashboard with WebSocket/SSE live updates surfacing evidence, hypotheses, and audit trail (Section 13)

Table 12. Sentinel-Prime capabilities mapped to the stated judging weighting.

17. Known Limitations and Future Work

17.1 Current Limitations

- Lab-scale prototype: real deployment requires integration with actual EDR, firewall, and IAM APIs in place of the current mocked containment actions.
- Honeypot placement strategy is currently heuristic; a future version could adopt a digital-twin/asset-graph-driven placement approach.
- Cross-agency, privacy-preserving federated threat-intelligence sharing is out of scope for the current design.
- OT/ICS decoys (Conpot) cover only a subset of real-world industrial protocols.
- Cloud/AWS telemetry and containment coverage is out of scope; the system targets on-premise lab infrastructure (Section 14).

17.2 Possible Extensions

Extension	Enhances	What It Does
Signal TTL / temporal decay	Correlation engine	Exponential decay on signal weights so stale signals fade from the unified threat score
Baseline cold-start fallback	Baseline store	Pre-computed global baselines for new or previously unseen entities
LLM fallback mode	Hypothesis / Analysis agent	Rule-based fallback reasoning when the Gemini Flash API is unreachable
Adaptive deception budget	Adaptive deception	Rate-limits concurrent decoy sets to prevent decoy flooding
IOC enrichment	Signal fusion	Cross-checks IPs and file hashes against AbuseIPDB / VirusTotal
Evidence preservation	Orchestrator	Automatic forensic snapshot before any destructive containment action
Campaign grouping	APT attribution	Clusters related entities into multi-incident campaign chains
CVE-feed-driven prioritisation	Vulnerability management	Extends the Cyber Entity Graph with live CVE and asset-topology context for the challenge's vulnerability-prioritisation build area

Table 13. Roadmap extensions beyond the current hackathon-stage implementation.

18. Conclusion

Sentinel-Prime's central architectural claim is narrow and testable: statistical detection, graph-based correlation, retrieval-grounded language-model reasoning, and execution authority can and should be kept as separate, independently auditable layers, with a deterministic policy gate as the sole authority over autonomous action on critical infrastructure. Every component described in this document — specialist-per-domain detectors, the Common Evidence Object boundary, the Cyber Entity Graph, the hybrid ATT&CK Graph-RAG, the Analysis/Critique/Action agent chain, adaptive deception as active hypothesis testing, and the deterministic risk engine and policy gate — exists to preserve that separation under the specific constraints of the Economic Times AI Cyber Resilience Hackathon: heterogeneous IT/OT telemetry, scarce labelled CNI-

specific data, and the requirement that autonomous response remain safe and auditable enough to trust on live infrastructure. The evaluation methodology in Section 15 defines how that claim will be tested once the benchmarking suite is run to completion.

References and Dataset Sources

- CSE-CIC-IDS2018 — https://www.unb.ca/cic/datasets/ids-2018.html ; AWS Open Data mirror: https://registry.opendata.aws/cse-cic-ids2018/
- LANL Authentication Dataset — https://csr.lanl.gov/data/auth/
- Splunk BOTS v3 — https://github.com/splunk/botsv3
- OTRF Security Datasets / Mordor — https://github.com/OTRF/Security-Datasets
- HAI ICS Dataset — https://github.com/icsdataset/hai
- MITRE ATT&CK STIX Data — https://github.com/mitre-attack/attack-stix-data
- [TODO: Insert Working Prototype link]
- [TODO: Insert Architecture Diagram export]
- [TODO: Insert Presentation Deck link]
- [TODO: Insert Demo Video link]